it. This causes the `extractData` function to fail because `http.get` doesn't work.

Imagine you're working on the program during this time when the HTTP service is down:

```
> AssertionError: "undefined" == "online"
```

This is a problem. Whenever the external site is down, you have no way to verify that the program is working correctly, even at the unit level. Unit tests shouldn't break when a dependency (in this case the external site) is unavailable. To make sure that unit tests work all the time, you need something that does the job of an HTTP service that isn't an HTTP service. You need something that won't break like an egg on your face. A thing that does the job during a test is a test double.

10.3.2. Test doubles

There are many terms for things that replace dependencies during a test (such as spies, mocks, and stubs). In this chapter *test double* or just *double* is used as a general term for something that replaces a dependency in a test. A double is like the stunt double for an actor or the friend who brought their jumper cables around that time your car battery was dead. When the real thing isn't suitable, you use a double.

In a minute you'll see how to create a double, but suppose for now you already have one; how do you get it into the program during the tests? By injecting it.

Injection

Picking up the actor metaphor again, consider a scene where an actor has to do a stunt: